TCP LARP: Fast Startup Through Speculative Pacing

Duong Dinh
Purdue University
West Lafayette, Indiana, USA
dinhd@purdue.edu

Ruibing Feng
Purdue University
West Lafayette, Indiana, USA
feng299@purdue.edu

***Abstract*—A new TCP connection must choose how much to send before receiving current delivery evidence. TCP LARP combines a bounded speculative launch with feedback-adaptive pacing and flight limits. Its 1 Gbps initial guess and 512-segment launch allowance are not permanent ceilings: qualified feedback permits ACK-driven window growth and delivery-gated rate extension. Delay growth, explicit congestion notification, and loss request relaxation. We implement LARP in Linux and evaluate it in two separate matched cohorts: 108 main observations against native CUBIC and BBR v1, and 300 observations against an experimental JumpStart-to-CUBIC port and native CUBIC. Main cells have three repetitions; nine initial calibration observations and a separate pilot are not pooled into performance means. Workloads include 1–512 MiB files, continuous demand, simultaneous aggressive senders, and staggered competitors across configured capacities from 25 Mbps to 5 Gbps. In the initial cohort, LARP delivers 955.71 Mbps on a 1 Gbps, 30 ms path and 1879.50 Mbps on a 2 Gbps, 5 ms path. In the extension, mean 128 MiB completion on the former path is 1.291 seconds for LARP and 3.158 seconds for the JumpStart–CUBIC port. The results characterize startup and larger-file behavior while exposing uneven sharing. They establish neither field-wide algorithmic novelty, equilibrium fairness, nor general Internet superiority.**

***Index Terms*—TCP congestion control, speculative pacing, bandwidth-delay product, flow completion time, network emulation**

I. Motivation and Introduction

An application that downloads an input object before starting its next computation waits for the object's last required byte. A new TCP connection has little current evidence about the rate at which the path can deliver that object. TCP slow start addresses this uncertainty by increasing the amount of outstanding data as acknowledgments (ACKs) return [1]. Increasing the initial window can reduce the feedback rounds needed for a short transfer; RFC 6928 describes an experimental initial window of ten segments [2]. A larger launch also permits more traffic before the sender learns whether the network can deliver it.

This tradeoff affects both the waiting application and other connections. On an idle path, a cautious launch can leave capacity unused. On an occupied path, an aggressive launch can create a queue, lose packets, or displace an incumbent. Faster newcomer completion therefore does not establish a better shared-network outcome. Completion, sustained delivery, and incumbent cost need separate measurements.

TCP LARP starts with a high pacing guess, then relaxes its rate when congestion feedback arrives. Throughout this paper, *LARP* denotes the official implementation with feedback-adaptive flight and rate limits. Its 1 Gbps initial target and 512-segment initial bound constrain speculative launch. They are not permanent operating ceilings: qualified delivery feedback permits an ACK-grown flight target and a moving probe bound that can exceed the initial rate. The sender receives no configured bottleneck capacity from the experiment.

Aggressive startup is established prior work. JumpStart paces an available initial batch over a handshake-derived round-trip time (RTT) [3]. Halfback adds ACK-clocked reverse-ordered proactive repair [4]. Delay-driven rate control and aggressive initialization also have antecedents. The question here concerns the behavior of a concrete TCP implementation: can a bounded speculative launch support short completion and useful later delivery without assuming that the launch guess equals capacity or available share?

The study contributes an explicit controller and recovery design, isolated real-kernel experiments, and measurements that separate application completion from delivery and sharing. An initial cohort contains 108 main observations and nine calibration observations comparing official LARP with native Linux CUBIC and BBR v1. A separate 300-observation cohort compares the same adaptive LARP with an experimental JumpStart-to-CUBIC port and native CUBIC across larger files and additional capacities. Both main cohorts use three repetitions per cell. Their aggregates remain separate; historical LARP variants and pilot observations are excluded. The paper does not claim field-wide novelty for starting fast and later reducing a rate.

Section II reviews prior mechanisms. Section III defines the problem. Section IV explains LARP, Section V describes the research methodology, and Section VI reports performance and its limits. Section VII concludes.

II. Related Work

A. Window Growth and Delivery Models

CUBIC specifies window growth for fast and long-distance paths, with a Reno-friendly region and congestion responses [5]. It is a native comparator here. Reno's slow start, congestion avoidance, and loss recovery provide background

for understanding ACK-driven growth [1]; standalone Reno is not a measured comparator in this cohort.

BBR constructs a delivery-bandwidth and propagation-delay model and uses pacing with distinct operating phases [6]. LARP also uses delivery and minimum RTT, but its initial target is a configured guess and its ongoing decisions use explicit delay episodes and congestion signals. The measured reference is the upstream Linux BBR v1 implementation. Results are not evidence about BBRv2 or BBRv3.

B. Fast Startup and Early Measurement

JumpStart uses a paced initial batch and then returns to ordinary TCP control [3]. Halfback follows a paced prefix with reverse-ordered proactive retransmissions and a fallback for longer transfers [4]. LARP retains ongoing rate relaxation and probing rather than handing later behavior to another controller. The supplementary comparison evaluates our JumpStart-style port with native CUBIC after its first ACK. It does not reproduce the authors' implementations or establish superiority over every startup method.

Paced Chirping uses increasing-rate trains separated by drain guards to probe early service [7]. Fast Bandwidth Estimation (FBE), described in *Small Chunks Can Talk*, estimates early bandwidth from ACK intervals and compensates for measurement distortion [8]. These approaches emphasize acquiring evidence early. LARP instead chooses a speculative rate before such evidence, then uses admitted delivery feedback to change its operating limits.

SUSS conditionally accelerates slow-start growth using ACK-train and RTT information, with clocking, pacing, and guard intervals [9]. HyStart++ addresses the exit decision through delay detection and a Conservative Slow Start phase [10]. LARP's delay-episode rule controls reductions and later probing throughout the connection. Neither SUSS nor HyStart++ is independently benchmarked here.

C. Delay Feedback and Aggressive Initialization

TIMELY adjusts rate using RTT and its gradient [11]. Swift uses an end-to-end delay target with additive increase and multiplicative decrease in a datacenter setting [12]. These mechanisms establish that host delay feedback is not new to LARP.

MLT is a particularly direct antecedent: its minimal rate controller starts at line rate and subsequently applies TIMELY-like RTT feedback [13]. Its transport supports bounded-loss gradient delivery and uses UDP data packets [13]. LARP retains TCP byte-stream reliability, ordinary receiver flow control, and Linux loss handling. Thus, the broad idea of fast initialization followed by relaxation is established. The present contribution is an implementation and a bounded performance/coexistence study, not a claim that the general idea is new.

SMaRTT combines delay, ECN, and optional trimming with rapid adjustment and explicit fairness mechanisms [14]. Careful Resume accelerates startup using retained path state, validation, and retreat when the estimate is inappropriate [15].

TABLE I
RELATED MECHANISMS AND EVALUATION BOUNDARY.

Method	Role in this study
CUBIC [5]	Window-based native comparator
BBR v1 [6]	Delivery-model native comparator
JumpStart [3]	Experimental CUBIC-handoff port measured separately
Halfback [4]	Proactive repair; literature comparison
Paced Chirping [7]	Early paced probing; literature comparison
FBE [8]	Early estimation; literature comparison
SUSS [9]	Conditional acceleration; literature comparison
HyStart++ [10]	Startup exit; literature comparison
TIMELY [11]	RTT-gradient control; literature comparison
Swift [12]	Delay-target control; literature comparison
MLT [13]	Line-rate initialization; literature comparison

These precedents also limit novelty claims for combining aggressive growth with congestion feedback. LARP instead begins from a configured guess without retained per-path history; this difference is a design choice whose benefits and costs require measurement.

Table I separates literature context from the measured references. Testing every related approach is unnecessary for explaining a mechanism, while a superiority claim over any one approach requires a matched comparison.

III. Problem to Solve

An initial safety bound and a learned operating limit serve different purposes. Let M be the maximum segment size in bytes. A permanently fixed flight window W imposes the approximate payload-rate bound

$$x \lesssim \frac{WM}{\mathrm{RTT}}. \tag{1}$$

For an illustrative $W = 512$, $M = 1448$ bytes and RTT of 30 ms, this is about 198 Mbps, even on a configured 1 Gbps path. Separately, a persistent 1 Gbps pacing ceiling would prevent filling a faster path. These are mathematical constraints, not alternate-controller performance observations. Official LARP treats these values as launch bounds and releases them after usable feedback.

The first research question is whether LARP sustains useful receiver delivery on both a larger BDP and a path faster than its initial guess. The second is whether finite completion changes between 1 and 128 MiB files. The third concerns incumbent cost: what does an established CUBIC or BBR flow deliver while LARP runs? A favorable solo result and an unfavorable sharing result can both answer these questions.

The design must preserve a bounded launch, avoid treating its own window-limited sample as proof of low capacity, obey ACK and recovery constraints, and respond to congestion after adaptation. Learning must mean permission to revise an estimate, not certainty that the path is uncongested or that the observed share is fair.

IV. Official LARP Design

A. Launch and Flight Target

Let R be the pacing target in bytes/s, R_0 its initial value, and $\widehat{T}$ the minimum observed RTT. Define

$$G = \max(\widehat{T}, 1\text{ ms}). \tag{2}$$

G spaces probes and repeated reductions and supplies the RTT fallback. It is a time interval, not a bandwidth gain. Initially $R = R_0 = 125{,}000{,}000$ bytes/s (1 Gbps), and the learned-feedback flag L is zero.

Set $L = 1$ after an admitted, nonzero delivery sample with valid RTT, outside loss recovery and without application limitation. The flag records permission to adapt. Define

$$W_{\text{cap}} = \begin{cases} 512, & L = 0, \\ W_{\text{kernel}}, & L = 1, \end{cases} \tag{3}$$

where W_{kernel} is the socket's congestion-window clamp. The target in segments is

$$W^*(R) = \min\left(W_{\text{cap}}, \max\left(4, \left\lceil \frac{2RG}{M} \right\rceil\right)\right). \tag{4}$$

The factor two supplies a rate–RTT flight budget. Because R may be speculative, this is not a measurement of the actual BDP. Using only achieved delivery to size the target could create a circular restriction: a small window would produce a low sample that kept the window small.

Initialization may select its target directly. Outside recovery, a later ACK obeys

$$W^+ = \max(1, \min(W^*, W + A)), \tag{5}$$

where A is newly acknowledged segments. A larger target therefore permits ACK-driven growth rather than an immediate jump. The receiver's advertised window and socket memory independently limit actual transmission.

B. Delivery Estimate and Delay Episodes

LARP uses Linux's per-ACK `rate_sample` interface [16]. Admission requires at least four delivered segments and an interval of at least $\max(\widehat{T}/8, 1\ \mu\text{s})$. Application-limited samples do not raise the delivery estimate. Two buckets, each approximately $2G$ long, retain a recent maximum $\widehat{B}$. Capacity-history aging pauses during loss repair because recovery can constrain delivery independently of the path. The minimum RTT itself does not expire.

Let D be a valid current RTT and

$$\theta = \max(500\ \mu\text{s}, \widehat{T}/8). \tag{6}$$

Delay is elevated when $D > \widehat{T} + \theta$. A delay event occurs on entering that condition or when RTT grows by another θ beyond the stored episode reference. The episode rearms only after RTT falls to at most $\widehat{T} + \theta/2$. An unchanged standing queue therefore does not request a new reduction on every ACK. This hysteresis does not imply that the queue or resulting share is harmless.

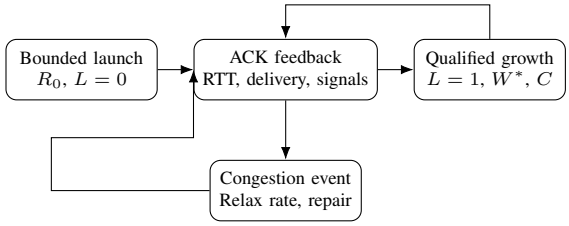


Fig. 1. Official LARP retains a bounded launch, then uses feedback to grow or relax. Congestion events take priority; window changes remain constrained by ACKs, receiver flow control, and recovery.

C. Rate Relaxation and Adaptive Probing

Congestion feedback has priority. For an accepted delay-only event,

$$R^+ = \max(R_{\min}, \min(R, \max(0.70R, 0.90\widehat{B}))), \tag{7}$$

where $R_{\min} = 125{,}000$ bytes/s (1 Mbps). ECN or loss instead permits a cut below the recent-delivery floor:

$$R^+ = \max(R_{\min}, 0.70R). \tag{8}$$

The first cut is immediate on feedback; later cuts are separated by at least G. A small delivery sample alone is not a congestion event.

An eligible probe requires elapsed G, application demand, no active loss recovery, and no congestion event taking priority. Its candidate rate is

$$P = \begin{cases} 1.05R, & \text{elevated delay}, \\ \max(1.25R, 1.02\widehat{B}), & \text{otherwise}. \end{cases} \tag{9}$$

Let E mean $L = 1$, an admitted nonzero sample on the current ACK, and no current elevated-RTT indication. The moving bound is

$$C = \begin{cases} \max(R_0, R, 1.25\widehat{B}), & E, \\ \max(R_0, R), & \text{otherwise}, \end{cases} \tag{10}$$
$$R^+ = \min(C, P).$$

A current admitted delivery sample without an elevated-RTT indication can thus support growth above the 1 Gbps initial guess. A missing RTT sample supplies no elevated-delay indication; it does not certify low queue delay. The bound moves with demonstrated recent delivery. Elevated delay cannot authorize extension beyond the current rate or launch guess. Including R in C prevents a smaller estimate from silently causing a rate cut. Integer arithmetic saturates rather than wrapping; equations omit rounding. Linux's maximum socket pacing rate remains authoritative.

D. Linux Integration and Recovery

The official congestion-control name is `larp`. The callbacks reside in `kernel/tcp_larp.c`; shared decision logic resides in `kernel/larp_core.h` [17]. The controller sets per-socket pacing and uses a sender `fq` queue. Linux continues to provide sequencing, ACK processing, flow control, and loss detection. Pacing requests interact with Linux's

TABLE II
EVALUATED PATHS. BUFFER UNITS ARE CAPACITY TIMES BASE RTT; ABSOLUTE FIFO CAPACITY THEREFORE CHANGES WITH BDP.

Path	Mbps	RTT (ms)	Buffer (BDP)	Queue
F	1000	5	1.00	FIFO
H	1000	30	1.00	FIFO
X	2000	5	1.00	FIFO
M	100	30	1.00	FIFO
S	25	90	0.25	RED/ECN

TABLE III
OFFICIAL EVALUATION: THREE REPETITIONS PER MAIN CELL, PLUS NINE SEPARATE CALIBRATION OBSERVATIONS.

Workload	Count
Solo finite: H/X, 1/128 MiB, three methods	36
Continuous send: H/X, three methods	18
1 MiB controls: F/S, three methods	18
LARP against CUBIC/BBR incumbent: H/M/S	18
128 MiB newcomer: H, three methods, two incumbents	18
Main total	108

transmission scheduler [18]; a requested rate is not a guarantee of immediate physical transmission.

A congestion-control callback must account for recovery windows. During the first recovery RTT LARP conserves packets using in-flight and newly acknowledged counts. Later recovery ACKs can grow the replacement window, bounded by W^*. Timeout recovery grows from Linux's collapsed window rather than freezing at one segment. ECN congestion-window reduction also respects the slow-start threshold. Loss/recovery entry, rather than every ACK marking a packet from an old flight, requests relaxation. Native regressions check these transitions, feedback qualification, above-launch growth, explicit-signal cuts, and saturation. They do not prove network fairness.

V. RESEARCH METHODOLOGY

A. Testbed and Controlled Conditions

The experiment runs real TCP inside one diskless ARM64 Linux guest with QEMU 11.1.2/HVF, two vCPUs, 4 GiB RAM, and kernel 6.12.115-0-virt. The matching kernel image and header-package version are pinned. Packages and source hashes are recorded. No host interfaces, kernel parameters, firewall settings, or installed packages are changed. The external guest NIC is disabled before traffic testing.

Five namespaces provide two independent senders, a bottleneck router, a delay router, and a receiver. Senders have separate `fq` queues with 1514-byte quantum and initial quantum. A token-bucket filter (TBF), with 16 KiB burst, shapes the forward bottleneck. Its child is a byte FIFO or Random Early Detection (RED) with ECN. Separate `netem` queues provide half the base RTT in each direction; reverse ACKs avoid the forward rate shaper. Virtual-link segmentation and receive offloads are disabled. Saved TCP metrics are disabled and each trial uses a fresh topology.

Table II lists the conditions. Relative to F, H changes RTT at the same configured capacity, and X changes capacity at the same RTT. Queue capacity scales with BDP, so these are not independent absolute-buffer sweeps. RED thresholds are one quarter and one half of its byte limit, with probability parameter 0.2. Guest socket-buffer ceilings are 16 MiB; the application requests a 4 MiB send buffer and a 16 MiB receive buffer. Buffer allocation and receiver-window values are retained in the traces.

CUBIC and upstream BBR v1 use their native kernel implementations, with identical traffic code, topology, and instrumentation. LARP is the official adaptive policy in every reported cell. No results from an older LARP version are included. JumpStart, Halfback, Reno, SUSS, and datacenter methods are not benchmarked in this initial cohort; the separate extension adds a JumpStart–CUBIC port. Calibration uses one continuous-send observation per method on F/H/X, yielding nine checks. Achieved native delivery, rather than the configured shaper alone, tests whether this guest can exceed 1 Gbps.

B. Workloads and Timing Boundaries

Table III accounts for the 108 main observations. Every cell has three repetitions, with algorithm order rotated within blocks. Files contain exactly 1 MiB (1,048,576 bytes) or 128 MiB (134,217,728 bytes); successful delivery must pass byte-count and SHA-256 checks. The larger file exposes behavior beyond launch. Different workload types remain separate aggregates.

For a finite file, data completion is

$$T_{\text{data}} = t_{\text{last payload read}} - t_{\text{connect start}}. \tag{11}$$

Sender and receiver share the guest monotonic clock. Connection setup is included; scheduled launch waits are excluded. Application completion, additionally including half-close and the receiver's validation reply, is retained separately. Finite trials have a 45 s wall limit after connect and 60 s socket-inactivity guard. Any noncompletion is retained as censored or failed and receives no completed-file time.

Continuous senders use two seconds of warmup, a common six-second receiver observation, and a half-second unmeasured tail. Pair incumbents start two seconds earlier; the common window begins two seconds after newcomer startup. Both senders continue through that window. In loaded finite trials, the incumbent stays active until the finite result or deadline. The harness verifies an established, alive incumbent with positive send queue before newcomer launch and at finite return.

C. Metrics and Integrity

Receiver goodput is payload bytes read during the common window divided by six seconds. We report both flows' rates under competition. Jain's index [19] is

$$J(\boldsymbol{x}) = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}. \tag{12}$$

An index near one indicates similar rates in that observation, not eventual convergence or equal demand satisfaction. Mean indices average per-trial indices; they are not recomputed from mean rates.

`TCP_INFO` sampling is 50 ms for finite transfers and 10 ms for continuous senders. It records cwnd, pacing, RTT, delivery, retransmissions, and unsent bytes. Queue snapshots bracket trials; receiver bins use 100 ms. Sampled maxima can miss brief excursions. Continuous application demand does not guarantee positive unsent bytes at every sample, particularly when socket memory is occupied by in-flight data; the observed backlog fraction is retained.

The checked reporter validates planned identities, rejects duplicates, checks successful payload counts and continuing incumbents, and verifies equality of ten controlled source files and package versions between calibration and main collection. Raw reports, source hashes, all statuses, and the exact official-subset selection are preserved [17]. Additional engineering diagnostics remain separate from the reported LARP/CUBIC/BBR aggregates. Means and observed minimum–maximum ranges describe three observations; they are not confidence intervals or significance tests.

D. JumpStart and Aggressive-Pair Extension

A separate 300-observation cohort retains the same LARP source and kernel. Comparators are native CUBIC and our `jumpcubic` port: it paces an available initial batch over the handshake RTT, then switches to native CUBIC on the first ACK [20]. Linux initializes CUBIC while retaining the current window and threshold. Native CUBIC recovery replaces the historical Reno port's first-flight repair bookkeeping after handoff. Each successful trial must record a successful kernel switch and final socket algorithm `cubic`. This is a specific experimental variant, not an original-author implementation.

The extension uses H/X/M/S from Table II, plus Z: 5 Gbps, 5 ms, one-BDP FIFO. Solo files are 1, 16, and 128 MiB on all five paths, with 512 MiB on X/Z. Other observations measure solo delivery, simultaneous LARP/LARP, LARP/JumpStart–CUBIC, JumpStart–CUBIC/self and CUBIC/self pairs, both mixed-flow incumbent orders, and two simultaneous 128 MiB files on H/X. Every main cell has three repetitions. The 13-observation pilot is excluded.

Continuous pairs use a ten-second common receiver window after two seconds of newcomer warmup; solo windows remain six seconds. Finite files finish independently, so a finite pair's overlap can end before both files complete. A finite deadline is $\min(180, \max(30, 3nS8/C + 10))$ seconds, where n is flow count, S bytes and C configured bits/s. Noncompletion remains explicit. Sampling is 50 ms finite and 20 ms continuous. Fresh neighbor state can inflate handshake RTT; the actual sample and receiver advertisement are retained. JumpStart's launch is limited by that advertisement, whereas LARP begins with its configured flight allowance. This is not a matched-initial-window ablation. Cohorts are never pooled into a common performance mean.

VI. Results and Discussion

A. Calibration and Learned Operating Limits

Calibration uses LARP, CUBIC and BBR on F/H/X before the main collection. On X, the native references deliver 1902.57 and 1912.62 Mbps, establishing that this guest can sustain more than the 1 Gbps launch guess. Table IV gives the main three-trial means. LARP delivers 955.71 Mbps on H and 1879.50 Mbps on X. Figure 2 shows observed ranges. These are achieved payload rates, rather than requested pacing values.

On H, LARP's maximum sampled window reaches 6210 segments. On X it reaches 2104 segments and the maximum sampled pacing target reaches 2432 Mbps. These traces confirm that the initial 512-segment and 1 Gbps values are not permanent ceilings. A pacing target above the bottleneck is still not delivery at that rate. Native comparisons and repeated receiver measurements are necessary to distinguish controller adaptation from a configured shaper target.

B. Finite Completion and File Size

Table V compares 1 MiB completion across the four finite-file paths. On F, LARP takes 28.62 ms, compared with 48.57 ms for CUBIC and 49.08 ms for BBR. On S, LARP takes 1289.00 ms, versus 892.67 and 1006.99 ms. The constrained path therefore remains part of the evaluation rather than being omitted when it does not favor LARP.

For 128 MiB, mean LARP completion is 1.288 s on H and 0.604 s on X. CUBIC takes 1.423 and 0.606 s; BBR takes 1.441 and 0.608 s. Figure 3 shows the observed ranges. The larger object exposes sustained delivery and recovery beyond launch. Comparisons here keep code, sampling and environment matched; they do not infer a universal crossover size or statistical ranking from three observations.

C. Continuing Competing Traffic

Table VI reports both receiver rates for incumbent-first pairs. On M with CUBIC, LARP delivers 93.58 Mbps while the incumbent delivers 2.44 Mbps. On S with CUBIC, the corresponding rates are 18.38 and 0.33 Mbps. These unequal shares show why a high LARP rate cannot alone establish successful coexistence. Delay-episode hysteresis and continued probing remain plausible contributors; this study does not isolate their individual effects.

With a BBR incumbent on H, LARP delivers 116.61 Mbps and the incumbent 678.13 Mbps in the common window. The result measures this arrival order and interval. It does not prove eventual equilibrium. The adaptation mechanism can learn beyond launch bounds while still providing unfavorable sharing or aggregate delivery under competition.

D. Larger Files Behind an Established Flow

Table VII measures 128 MiB newcomers on H while CUBIC or BBR remains active. Completion counts accompany the mean of completed files; an incomplete observation is not replaced by the wall deadline. This workload differs from a fixed-duration overlap that can end before the newcomer

TABLE IV
OFFICIAL LARP VERSUS NATIVE REFERENCES: MEAN RECEIVER DELIVERY AND SOLO 128 MIB COMPLETION, THREE OBSERVATIONS PER CELL. H IS 1 GBPS/30 MS; X IS 2 GBPS/5 MS. FULL RANGES AND TRACES ARE PRESERVED.

Method	H Mbps	X Mbps	H 128 MiB (s)	X 128 MiB (s)
LARP	955.71	1879.50	1.288	0.604
CUBIC	954.78	1912.54	1.423	0.606
BBR v1	955.00	1912.35	1.441	0.608

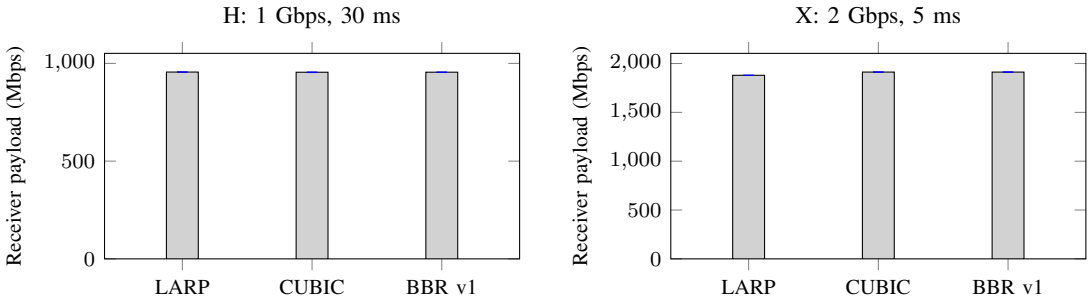


Fig. 2. Continuous-send receiver delivery after warmup. Bars are three-trial means; whiskers are observed minimum and maximum, not confidence intervals.

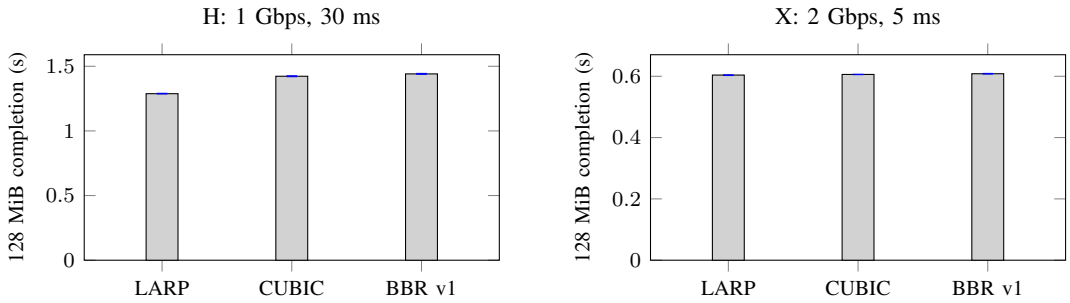


Fig. 3. 128 MiB receiver completion, including connect. Means and observed ranges use three successfully checked files per cell.

TABLE V
MEAN 1 MIB DATA COMPLETION, MILLISECONDS; THREE OBSERVATIONS PER CELL. LOWER IS BETTER.

Method	F	H	X	S
LARP	28.62	154.68	27.25	1289.00
CUBIC	48.57	271.82	47.40	892.67
BBR v1	49.08	291.89	49.14	1006.99

TABLE VI
INCUMBENT-FIRST PAIRS: MEAN MBPS FOR EACH FLOW AND MEAN PER-TRIAL JAIN INDEX. THREE OBSERVATIONS PER ROW.

Path	Incumbent	LARP	Incumbent	J
H	CUBIC	682.85	270.33	0.842
H	BBR	116.61	678.13	0.667
M	CUBIC	93.58	2.44	0.526
M	BBR	54.78	42.56	0.983
S	CUBIC	18.38	0.33	0.518
S	BBR	0.89	23.09	0.538

finishes. Incumbent checks verify that competition persists through the recorded result.

TABLE VII
H: MEAN 128 MIB COMPLETION (SECONDS) WITH A CONTINUING INCUMBENT; COMPLETED COUNT IN PARENTHESES. MEANS EXCLUDE CENSORED FILES.

Newcomer	CUBIC incumbent	BBR incumbent
LARP	2.794 (3/3)	8.241 (3/3)
CUBIC	5.399 (3/3)	censored (0/3)
BBR v1	2.162 (3/3)	2.605 (3/3)

All three CUBIC newcomers behind BBR reach the configured wall limit. LARP completes all three in 8.241 s on average, compared with 2.605 s for a BBR newcomer. With CUBIC incumbent, the corresponding LARP and BBR means are 2.794 and 2.162 s. There are 3 noncompleted official observations in the full main selection. All statuses and any receiver checkpoints remain in the raw report. A lower newcomer completion time is interpreted alongside Table VI, because displacement of an incumbent can improve newcomer timing without improving shared delivery.

TABLE VIII

EXTENSION: MEAN SOLO COMPLETION IN SECONDS, THREE OBSERVATIONS PER CELL. JS–C DENOTES OUR JUMPSTART-TO-NATIVE-CUBIC PORT. CONDITIONS H/X/M/S FOLLOW TABLE II; Z IS 5 GBPS/5 MS, ONE-BDP FIFO. FULL RANGES AND COMPLETION COUNTS ARE RETAINED IN THE CHECKED REPORT.

Path	MiB	LARP	JumpStart–CUBIC	CUBIC
H	1	0.154	0.479	0.272
H	16	0.309	1.752	0.443
H	128	1.291	3.158	1.425
X	1	0.027	0.087	0.048
X	16	0.104	0.317	0.115
X	128	0.603	0.817	0.607
X	512	2.321	2.500	2.291
Z	1	0.027	0.086	0.048
Z	16	0.094	0.315	0.084
Z	128	0.344	0.617	0.302
Z	512	1.296	1.557	0.952
M	1	0.212	0.478	0.290
M	16	1.514	2.119	1.613
M	128	11.344	11.915	11.432
S	1	1.956	1.403	0.895
S	16	8.158	9.226	8.387
S	128	58.538	56.126	55.388

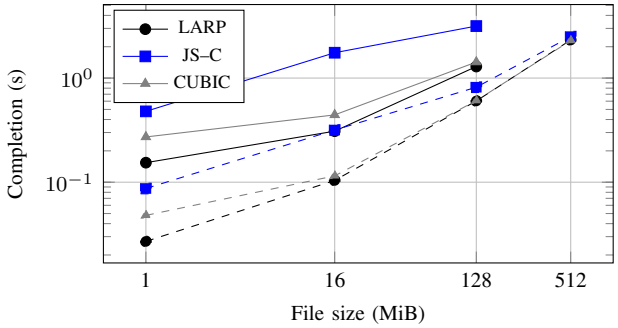


Fig. 4. Extension: mean completion versus file size. Solid lines: H (1 Gbps/30 ms), through 128 MiB. Dashed: X (2 Gbps/5 ms), through 512 MiB. Colors identify methods; axes are logarithmic. Each point averages three observations; full ranges are retained.

E. Larger Files and JumpStart with CUBIC

The extension keeps the adaptive LARP controller unchanged and evaluates a specific JumpStart–CUBIC port. Table VIII reports solo completion. On H, 128 MiB takes 1.291 s for LARP, 3.158 s for JumpStart–CUBIC, and 1.425 s for native CUBIC. On Z, 512 MiB takes 1.296, 1.557, and 0.952 s, respectively. Native CUBIC sustained delivery on Z is 4769.54 Mbps; the configured 5 Gbps is therefore treated as a stress setting, not a physical throughput guarantee. Figure 4 shows the size dependence within this cohort.

The comparison does not isolate a unique adaptive-control contribution. JumpStart's first batch is limited by the initial receiver advertisement and its CUBIC handoff retains that startup window and threshold. LARP has a different configured flight allowance. A similarly paced, matched-window CUBIC startup and component ablations remain necessary to separate aggressive launch from ongoing feedback-policy effects. No result is attributed to the original JumpStart implementation or to every possible CUBIC integration.

TABLE IX

EXTENSION: SIMULTANEOUS CONTINUOUS PAIRS, MEAN RECEIVER MBPS AND MEAN PER-TRIAL JAIN INDEX OVER A TEN-SECOND COMMON WINDOW AFTER WARMUP. THREE OBSERVATIONS PER ROW; BOTH FLOWS REMAIN ACTIVE. JS–C IS JUMPSTART–CUBIC.

Path	A / B	A Mbps	B Mbps	J
H	LARP / LARP	489.68	459.55	0.998
H	LARP / JS–C	758.55	195.65	0.742
H	JS–C / JS–C	477.28	475.46	1.000
X	LARP / LARP	956.98	950.17	1.000
X	LARP / JS–C	1015.83	896.75	0.996
X	JS–C / JS–C	1008.41	903.90	0.985
Z	LARP / LARP	2718.24	1985.73	0.966
Z	LARP / JS–C	2317.29	2442.44	0.892
Z	JS–C / JS–C	2269.53	2477.88	0.822
M	LARP / LARP	48.72	49.88	0.989
M	LARP / JS–C	93.19	2.35	0.525
M	JS–C / JS–C	49.97	45.66	0.990
S	LARP / LARP	6.25	12.38	0.503
S	LARP / JS–C	18.34	0.30	0.516
S	JS–C / JS–C	9.26	9.53	0.998

TABLE X

EXTENSION: TWO SIMULTANEOUS 128 MIB FILES, MEAN COMPLETION IN SECONDS. EACH FILE FINISHES INDEPENDENTLY; THE SURVIVING FILE CAN USE RELEASED CAPACITY. THESE ARE COMPLETION MEASUREMENTS, NOT SUSTAINED-FAIRNESS ESTIMATES.

Path	A / B	A seconds	B seconds
H	LARP / LARP	2.437	2.300
H	LARP / JS–C	1.321	3.437
X	LARP / LARP	1.149	1.140
X	LARP / JS–C	0.918	1.247

F. Two Aggressive Senders

Table IX measures simultaneous continuous senders. Identical algorithms are not assumed to converge to equal shares. On M, the LARP/JumpStart–CUBIC pair delivers 93.19/2.35 Mbps. On S, two LARP flows deliver 6.25/12.38 Mbps. Across-run means can conceal opposite winners in individual repetitions; the table therefore also reports mean per-trial Jain indices, not an index computed from averaged rates. Three observations and a ten-second window characterize these cases; they do not establish equilibrium fairness or the cause of each imbalance. Both mixed-flow incumbent orders and the CUBIC/self control remain in the checked report [20].

Table X separates finite pair completion from persistent competition. Faster completion can shorten the overlap itself. Consequently, a favorable solo or finite-pair result is not a substitute for both-flow delivery, delay and retransmission analysis.

G. Limits of Interpretation

The two guest CPUs and receiver share scheduling. A configured shaper rate is not a physical-link guarantee, although native calibration verifies delivery above the initial guess in this environment. TBF bursting, disabled offloads,

socket memory, and the sample interval influence the observed conditions. File-size, capacity, RTT and queue comparisons are bounded to the stated cells; buffer bytes scale with BDP. The study does not establish a universal crossover size.

Three repetitions are insufficient for tail percentiles or equilibrium fairness. The first cohort uses incumbent-first pairs; the extension adds simultaneous starts, self competition, and both mixed-flow incumbent orders. Route changes, many-flow incast, physical links, and BBRv2/v3 remain untested. Minimum RTT does not expire: a longer route can appear congested relative to an obsolete baseline, while an initially queued handshake can inflate the baseline. An unchanged elevated RTT still permits five-percent probes; missing current RTT can permit above-launch extension after learning. These are properties of the measured controller, not fixes claimed by this study. Delivery history and probing can preserve an aggressive share. Isolating each rule and testing bounded probe/evaluate/drain alternatives remain future work.

VII. Conclusion

TCP LARP combines bounded speculative startup with feedback-adaptive pacing and flight limits. The Linux implementation can operate beyond its initial 1 Gbps and 512-segment launch values. Separate cohorts compare it with native CUBIC and BBR v1 and with an experimental JumpStart–CUBIC port. Larger files, capacities through a configured 5 Gbps, and simultaneous aggressive pairs extend the evaluation beyond short solo transfers. On the extension's 1 Gbps/30 ms path, mean 128 MiB completion is 1.291 s for LARP and 3.158 s for the JumpStart–CUBIC port.

Useful solo delivery and shorter completion do not establish balanced sharing. The measurements concern this implementation, its startup budgets, and one virtual testbed. Future work should isolate launch versus ongoing-policy effects, evaluate bounded probe/evaluate/drain rules, require appropriate RTT freshness, revalidate changing path baselines, and test longer many-flow and physical-network settings. Fast initialization and delay-based adaptation have established precedents; the contribution is a reproducible implementation and a bounded analysis of its benefits and costs.

References

[1] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," RFC Editor, Tech. Rep. RFC 5681, Sep. 2009. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5681

[2] J. Chu, N. Dukkipati, Y. Cheng, and M. Mathis, "Increasing TCP's Initial Window," RFC Editor, Tech. Rep. RFC 6928, Apr. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6928

[3] D. Liu, M. Allman, S. Jin, and L. Wang, "Congestion control without a startup phase," in *Proc. Workshop on Protocols for Fast Long-Distance Networks (PFLDnet)*, Feb. 2007. [Online]. Available: https://pfld.net/2007/paper/WithoutStartup.pdf

[4] Q. Li, M. Dong, and P. B. Godfrey, "Halfback: Running short flows quickly and safely," in *Proc. 11th ACM Conf. Emerging Networking Experiments and Technologies (CoNEXT)*, 2015. [Online]. Available: https://conferences2.sigcomm.org/co-next/2015/img/papers/conext15-final151.pdf

[5] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, "CUBIC for Fast and Long-Distance Networks," RFC Editor, Tech. Rep. RFC 9438, Aug. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9438

[6] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *Communications of the ACM*, vol. 60, no. 2, pp. 58–66, Feb. 2017. [Online]. Available: https://research.google/pubs/bbr-congestion-based-congestion-control/

[7] J. Misund and B. Briscoe, "Paced chirping: Rapid flow start with very low queuing delay," in *Proc. IEEE Conf. Computer Communications Workshops (INFOCOM WKSHPS)*, 2019, pp. 798–804. [Online]. Available: https://bobbriscoe.net/projects/latency/paced_chirping_brief_ieeegi19.pdf

[8] L. Li, Y. Chen, and Z. Li, "Small chunks can talk: Fast bandwidth estimation without filling up the bottleneck link," in *Proc. IEEE/ACM 31st Int. Symp. Quality of Service (IWQoS)*, 2023, pp. 1–10. [Online]. Available: https://doi.org/10.1109/IWQoS57198.2023.10188775

[9] M. Arghavani, H. Zhang, D. Eyers, and A. Arghavani, "SUSS: Improving TCP performance by speeding up slow-start," in *Proc. ACM SIGCOMM 2024 Conf.*, 2024, pp. 151–165. [Online]. Available: https://cs.stanford.edu/~keithw/sigcomm2024/sigcomm24-final280-acmpaginated.pdf

[10] P. Balasubramanian, Y. Huang, and M. Olson, "HyStart++: Modified Slow Start for TCP," RFC Editor, Tech. Rep. RFC 9406, May 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9406

[11] R. Mittal, V. T. Lam, N. Dukkipati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, and D. Zats, "TIMELY: RTT-based congestion control for the datacenter," in *Proc. ACM SIGCOMM*, 2015, pp. 537–550.

[12] G. Kumar, N. Dukkipati, K. Jang, H. Wassel, X. Wu, B. Montazeri, Y. Wang, K. Springborn, C. Alfeld, M. Ryan, D. Wetherall, and A. Vahdat, "Swift: Delay is simple and effective for congestion control in the datacenter," in *Proc. ACM SIGCOMM*, 2020, pp. 514–528.

[13] H. Wang, H. Tian, J. Chen, X. Wan, J. Xia, G. Zeng, W. Bai, J. Jiang, Y. Wang, and K. Chen, "Towards domain-specific network transport for distributed DNN training," in *Proc. USENIX NSDI*, 2024, pp. 1421–1443. [Online]. Available: https://www.usenix.org/conference/nsdi24/presentation/wang-hao

[14] T. Bonato, A. Kabbani, A. Ghalayini, A. Agarwal *et al.*, "SMaRTT: Sender-based marked rapidly-adapting trimmed & timed transport," 2026, arXiv:2404.01630, version 4; first version 2024. [Online]. Available: https://arxiv.org/abs/2404.01630

[15] N. Kuhn, E. Stephan, G. Fairhurst, R. Secchi, and C. Huitema, "Careful Resume: Convergence of Congestion Control from Retained State," RFC Editor, Tech. Rep. RFC 9959, May 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9959.html

[16] Linux kernel contributors, "TCP delivery-rate estimator, Linux v6.12, net/ipv4/tcp_rate.c," 2024, source code, accessed Oct. 30, 2026. [Online]. Available: https://github.com/torvalds/linux/blob/v6.12/net/ipv4/tcp_rate.c

[17] TCP LARP project, "TCP LARP implementation and experimental artifacts," 2026, official adaptive LARP implementation and November 4 experiments; raw source hashes and checked reports. [Online]. Available: https://github.com/duongddinh/tcplarp_priv/tree/research/adaptive-startup-head-to-head

[18] Linux kernel contributors, "TCP output and earliest-departure pacing, Linux v6.12, net/ipv4/tcp_output.c," 2024, source code, accessed Oct. 30, 2026. [Online]. Available: https://github.com/torvalds/linux/blob/v6.12/net/ipv4/tcp_output.c

[19] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Tech. Rep. DEC Research Report TR-301, Sep. 1984. [Online]. Available: https://www.cse.wustl.edu/~jain/papers/fairness.htm

[20] TCP LARP project, "Adaptive LARP and JumpStart with native CUBIC: matched experiments," 2026, november 5 cohort; source hashes and all outcomes retained. [Online]. Available: https://github.com/duongddinh/tcplarp_priv/tree/research/adaptive-startup-head-to-head/results